UCC
University College Cork, Ireland
Coláiste na hOllscoile Corcaigh

# Workplace Technology and Skills (CS3306)

## Linux Operating System: Overview and Fundamentals

Lecture 04

Dr Md Noor-A-Rahim

January 15, 2026

# What is Linux?



## Linux Distributions

A free and open-source operating system kernel combined with GNU utilities to create complete operating systems known as **Linux distributions**.

**Key Characteristics:**

- **Open Source**: Source code is freely available
- **Free**: No licensing costs
- **Secure**: Transparent code enables rapid security fixes
- **Customizable**: Modify to suit specific needs
- **Community-Driven**: Developed by thousands of contributors worldwide

# Brief History

| Year | Milestone |
|---|---|
| **1991** | Linus Torvalds creates Linux kernel as a hobby project |
| **1994** | Linux 1.0 released |
| **2000s** | Enterprise adoption (e.g., Red Hat) |
| **2008** | Android launches (Linux-based) |
| **Present** | Dominates servers, cloud, mobile, supercomputers |

# Why Learn Linux? - Industry Dominance

| Sector | Linux Market Share |
|---|---|
| **Web Servers** | 96% of top 1 million servers |
| **Cloud Infrastructure** | 90% of public cloud workloads |
| **Supercomputers** | 100% of TOP500 supercomputers |
| **Mobile Devices** | 70%+ (Android is Linux-based) |
| **IoT Devices** | Majority of embedded systems |

# Why Learn Linux? - Career Opportunities

**High-Demand Roles Requiring Linux:**

- **DevOps Engineer**: CI/CD pipelines, containerization
- **System Administrator**: Server management, automation
- **Cloud Architect**: AWS, Azure, GCP all use Linux
- **Cybersecurity Analyst**: Security tools run on Linux
- **Data Engineer**: Big data tools (Hadoop, Spark) run on Linux

# Why Learn Linux? - Technical Advantages

**Stability:**

- Servers can run for years without rebooting
- Less prone to crashes

**Security:**

- Permissions model prevents unauthorized access
- Rapid security updates

**Performance:**

- Efficient resource usage
- Lightweight

**Flexibility:**

- Complete control over the system
- Customize everything

# Why Learn Linux? - Financial Benefits

University College Cork, Ireland
Coláiste na hOllscoile Corcaigh

## Cost Comparison (100-server environment)

```
Windows Server:
  100 licenses x $1,000 = $100,000

Linux:
  100 installations x $0 = $0
```

# Linux Distributions (Distros)

**What is a Distribution?**

- Linux Kernel + GNU Utilities + Package Manager + Desktop Environment

**Popular Distributions:**

| Distribution | Target Use Case | Package Manager |
|---|---|---|
| **Ubuntu** | Beginners, Desktops | APT (.deb) |
| **Debian** | Servers, Stability | APT (.deb) |
| **RHEL/AlmaLinux** | Enterprise Servers | YUM/DNF (.rpm) |
| **Fedora** | Developers, Latest Features | DNF (.rpm) |
| **Arch Linux** | Advanced Users | Pacman |
| **Kali Linux** | Security Testing | APT (.deb) |
| **Raspberry Pi OS** | IoT, Education | APT (.deb) |

# The Terminal

**Terminal** is a text-based interface to interact with the computer.

**Terminal Prompt**

```
user@hostname:~$
```

## Why Use Terminal Instead of GUI?

| Advantage | Explanation |
|---|---|
| **Speed** | Faster for many tasks once learned |
| **Lightweight** | Consumes fewer system resources |
| **Remote Access** | SSH allows remote server management |
| **Universal** | Same commands across all Linux systems |
| **Automation** | Scripts can automate repetitive tasks |

# The Shell

**Shell** is a program that interprets your commands and executes them.

User → Types command → Terminal → Shell → OS → Kernel

**Common Shells:**

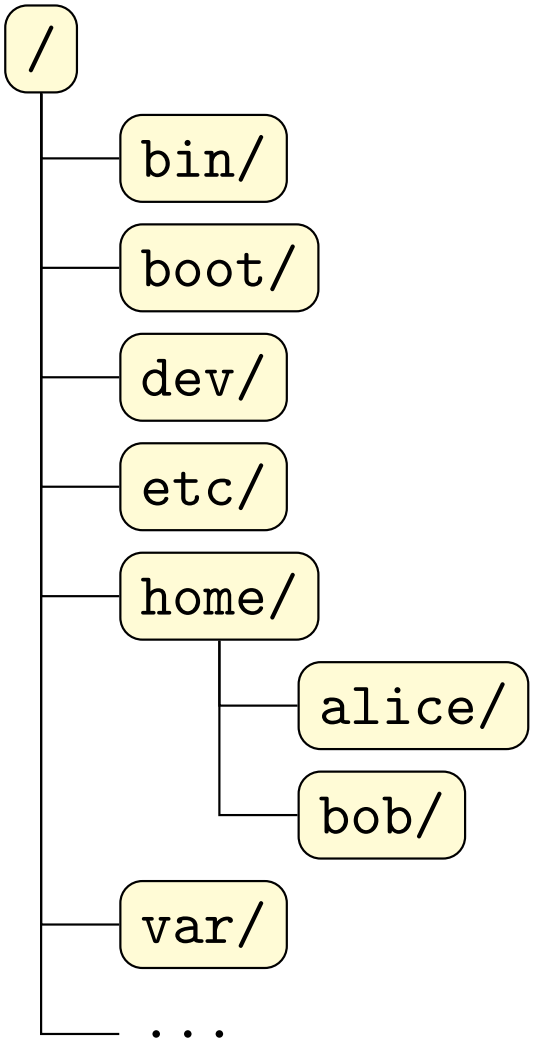
- **bash** (Bourne Again Shell): Most popular, default on most systems.
- **zsh** (Z Shell): Advanced features, popular with developers.
- **sh** (Bourne Shell): Original Unix shell.

## Check your shell

```
echo $SHELL
# Output: /bin/bash
```

# Linux File System Hierarchy

Everything in Linux starts from / (root directory).



# Path Types

**Absolute Path** (starts with /):

```
/home/alice/Documents/report.txt
```

**Relative Path** (relative to current location):

```
Documents/report.txt
./Documents/report.txt
```

**Special Symbols:**

- / : Root directory
- ~ : Home directory
- . : Current directory
- .. : Parent directory

# Navigation Commands

```
# Print Working Directory
pwd

# List files
ls                      # List files
ls -l                   # Long format
ls -la                  # Include hidden files

# Change Directory
cd /etc                 # Go to /etc
cd ~                    # Go to home directory
cd ..                   # Go up one level
cd -                    # Go to previous directory
```

# Information Commands

```
# Show current user
whoami

# Show system information
uname -a

# Display message
echo "Hello, Linux!"

# Show current date and time
date

# Getting Help
man ls                  # Manual pages
ls --help               # Quick help
```

# User-Defined Variables

```
# Assign value (NO SPACES around =)
NAME="Alice"
AGE=25

# Use variables (with $)
echo $NAME
# Output: Alice

echo "My name is $NAME and I am $AGE years old"
```

**Using Braces:**

```
FILE="report"
echo "${FILE}.txt"    # Output: report.txt
```

# Environment Variables

**Environment Variables** are passed to child processes.

```
# View variables
env
echo $HOME
echo $USER
```

**Common Variables:**

- `HOME`: User's home directory
- `PATH`: Directories to search for commands
- `USER`: Current username
- `SHELL`: Current shell

# Exporting User-Defined Variables

```
# Step 1: Create a shell variable
myvar="hello there"
echo $myvar
# Output: hello there

# Step 2: Start a child process (new shell)
bash

# Step 3: Try to access 'myvar' in the child shell
echo $myvar
# Output: (nothing, variable not found)

# Step 4: Exit the child shell
exit
```

# Exporting User-Defined Variables

```
# Step 5: Export 'myvar' as an environment variable
export myvar="hello there"

# Step 6: Start a new child process (new shell)
bash

# Step 7: Access 'myvar' in the child shell
echo $myvar
# Output: hello there

# Step 8: Exit the child shell
exit
```

# Secure Shell (SSH)

**SSH** is a cryptographic network protocol for secure remote access.

```
# Connect to remote server
ssh username@hostname

# Run command on remote server
ssh username@hostname 'ls -l'
```

## Quick Exercise: Access to UCC CS server

**From UCC network:**

`ssh userID@cs1dev.ucc.ie`

**From outside UCC network:**

1. `ssh userID@csgate.ucc.ie`
2. `ssh userID@cs1dev.ucc.ie`

# Secure File Copy (SCP)

```
# Copy file TO remote server
scp myfile.txt username@hostname:/path/to/dest/

# Copy file FROM remote server
scp username@hostname:/path/to/file.txt .

# Copy directory recursively
scp -r myfolder/ username@hostname:/path/
```

# Additional Resources

- **Bash Guide**: `https://www.gnu.org/software/bash/manual/`
- **Book**: "Learning the bash Shell" by Cameron Newham

# Thanks!